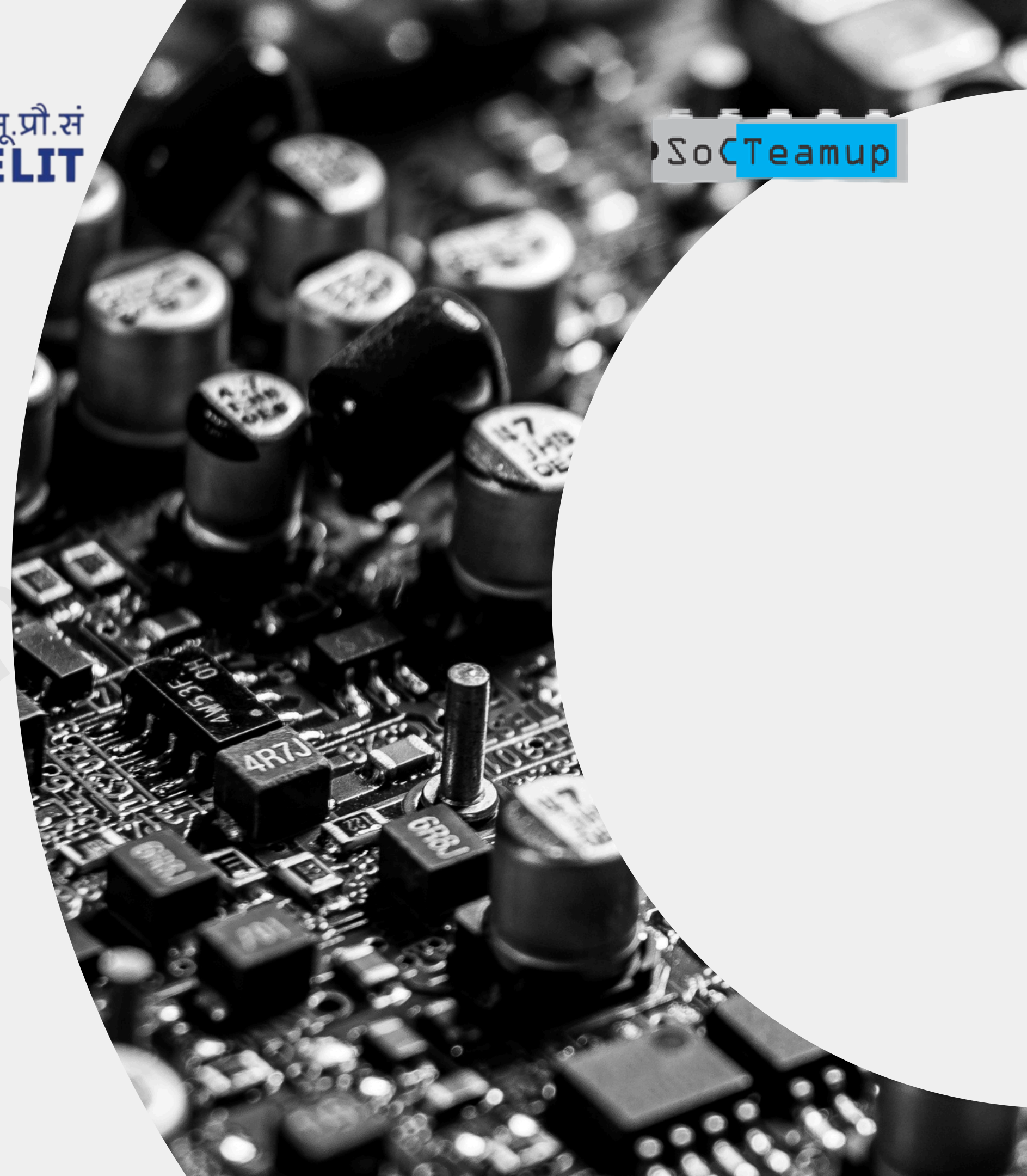


Mapping Verilog constructs to digital components





Outline

What is RTL mapping?
assign → logic gates
always @(*) → combinational logic
always @(posedge clk) → flip-flops
if / case → mux and decoder
Blocking vs Non-blocking
Latch inference and how to avoid it
Loop unrolling
FSM mapping to hardware
Register array and memory
Parameterized modules
RTL vs netlist code comparison
Yosys synthesis insight
Best coding practices for synthesis
Summary

What is RTL Mapping?

RTL (Register Transfer Level) describes how data moves between registers using logic.

During mapping synthesis tool translates RTL into:

- Logic gates (AND, OR, NOT)
- Flip-flops
- Multiplexers
- Memory blocks

Tools used: Yosys (synthesis), Verilator (simulation)

➔ assign → Logic Gates

```
assign y = a & b;  
assign z = ~y;
```

Inferred Hardware:

- AND gate for a & b
- NOT gate for ~y

Yosys Output:

Generated \$and cell
Generated \$not cell

Concept: assign creates purely combinational logic (no clock)

➔ **always @(*)** → **Combinational Logic**

```
always @(*) begin
    if (sel)
        out = a;
    else
        out = b;
end
```

Mapped to:
2:1 multiplexer

- This is functionally same as:

```
assign out = sel ? a : b;
```

- **Example** : 2:1 multiplexer

➔ **always @(posedge clk) → Flip-Flops**

```
always @(posedge clk)  
  q <= d;
```

Mapped to:

- D flip-flop

Tool infers synchronous registers

Add reset logic:

```
if (rst) q <= 0;
```

- Inferred DFF with synchronous reset

Latch Inference & How to Avoid

Latch inference occurs when synthesis tools automatically create latches (level-sensitive storage) from incomplete combinational logic instead of pure gates.

```
always @(*) begin
if (en)
q = d; // latch inferred if 'else' is missing
end
```

- Fix: Cover all conditions!

```
always @(*) begin
if (en)
q = d; // latch inferred if 'else' is missing
else q=0;
end
```

- Synthesis infers level-sensitive latch if not fully defined and it is usually avoided

➔ if / case → Muxes or Decoders

- Useful for: ALUs, Bus selectors
- Mapped to: 4:1 multiplexer, OR series + AND selectors

```
case (sel)
  2'b00: y = a;
  2'b01: y = b;
  2'b10: y = c;
default: y = d;
endcase
```

➔ Blocking vs Non-blocking Assignments

```
// Combinational (Blocking)
always @(*) begin
    a = b;
    c = a;
end
```

```
// Sequential (Non-blocking)
always @(posedge clk) begin
    a <= b;
    c <= a;
end
```

- Blocking (=): used for combinational.
- Non-blocking (<=): used for clocked logic.
- Using wrong type may create mismatches during synthesis.

➔ Loops Unrolling Hardware Replication

- Loop unrolling in RTL synthesis expands for loops during elaboration, creating multiple hardware instances instead of sequential iterations.

```
genvar i;  
generate  
  for (i = 0; i < 4; i++) begin : mux_array  
    assign out[i] = sel ? a[i] : b[i];  
  end  
endgenerate
```

- Tool unrolls loop into 4 AND gates
→ No looping logic in hardware
- Use generate block for scalable modules

→ FSM Mapping to hardware

Synthesis tools convert behavioral FSM code into actual gates through these steps:

State Storage:

- `reg [1:0]` state becomes 2 D flip-flops (one per bit)
- Each FF captures next-state value on posedge `clk`

Next-State Logic:

- `state <= state + 1` becomes adder + multiplexer
- When `en=1`: ADD computes `state+1` → selects via MUX
- When `en=0`: MUX selects current state (hold)

Reset Handling:

- if (`reset`) sets both FFs async to 0 (`RESET` pin active-high)

Output Generation:

- `state_out = state` → direct wire connection (no extra logic)

Register Arrays and Memories

Synthesis tools analyze your RTL code patterns to decide between flip-flop registers or RAM blocks:

Registers are inferred when:

- Small arrays (≤ 32 words) like `reg [7:0] rf[0:3];`
- Multiple read/write ports needed in same cycle
- Hardware: Individual DFF cells scattered across design (uses more area, multi-port capable)

Memory/RAM is inferred when:

- Large arrays (> 64 words) like `reg [7:0] mem[0:255];`
- Single block with address/data ports: always `@(posedge clk)`
`if(wen) mem[addr] <= din;`
- Hardware: Dedicated RAM macro (single/dual-port, saves area)

➔ Parameterization = Scalable Hardware

- **Scalable Design:** Single module handles different bus widths via parameter WIDTH=8
- **Instantiation Override:** `#(.WIDTH(16)) inst_name(...)` customizes at use time
- **Constant Propagation:** Parameters become constants during elaboration for optimization
- **Generate Compatibility:** Works with for/if generate for repetitive/conditional logic
- **Reusability:** Write once, use everywhere with different configurations (ALU widths, FIFO depths)

```
module mux #(parameter WIDTH = 8) (  
    input [WIDTH-15:0] a, b,  
    input sel,  
    output [WIDTH-15:0] y  
);  
    assign y = sel ? b : a;
```

- WIDTH = 4 → 4 muxes
- WIDTH = 16 → 16 muxes
- Reuse with different configurations

➔ RTL vs Synthesized Netlist (Yosys)

RTL Netlist

```
assign sum = a & b; //  
Please add these numbers  
always @(posedge clk)  
q <= d; // Please store this  
on clock edge
```

Tools understand intent
and create optimal logic
automatically.

vs

Synthesized Netlist

```
AND2X1 u1 (.A(a0), .B(b0),  
.Y(n3)); // Specific sky130  
gate  
DFFX1 u2 (.D(n3), .CLK(clk),  
.Q(q)); // Specific flip-flop
```

Every wire connection
and gate type is spelled
out for place & route.

Best Practices for Predictable Mapping

-  Use non-blocking for clocked logic
-  Avoid latches by covering all conditions
-  Don't use ``#delay` or `$display` in synthesizable code`
-  Use parameters and generate blocks for scalability
-  Simulate with Verilator before synthesis
-  Review netlist using Yosys ``show` and logs`

Summary

Verilog constructs map to gates, muxes, flops

Tools infer logic from behavioral code

Blocking and Non-blocking assignments

FSM Mapping, Register arrays and memories

RTL vs Synthesis netlist

Thank You

echiphu

